第14章 类型信息

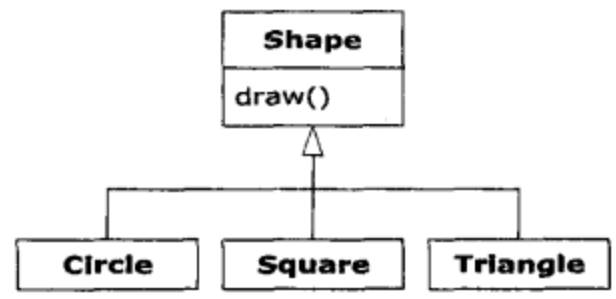
运行时类型信息使得你可以在程序运行时发现和使用类型信息。

它使你从只能在编译期执行面向类型的操作的禁锢中解脱了出来，并且可以使用某些非常强大的程序。对RTTI的需要，揭示了面向对象设计中许多有趣（并且复杂）的问题，同时也提出了如何组织程序的问题。

本章将讨论 Java 是如何让我们在运行时识别对象和类的信息的。主要有两种方式：一种是“传统的”RTTI，它假定我们在编译时已经知道了所有的类型；另一种是“反射”机制，它允许我们在运行时发现和使用类的信息。

14.1 为什么需要RTTI

下面看一下我们已经很熟悉了的一个例子，它使用了多态的类层次结构。最通用的类型（泛型）是基类**Shape**，而派生出的具体类有**Circle**、**Square**和**Triangle**（见右图所示）。



这是一个典型的类层次结构图，基类位于顶部，派生类向下扩展。面向对象编程中基本的目的是：让代码只操纵对基类（这里是**Shape**）的引用。这样，如果要添加一个新类（比如从**Shape**派生的**Rhomboid**）来扩展程序，就不会影响到原来的代码。在这个例子的**Shape**接口中动态绑定了**draw()**方法，目的就是让客户端程序员使用泛化的**Shape**引用来调用**draw()**。**draw()**在所有派生类里都会被覆盖，并且由于它是被动态绑定的，所以即使是通过泛化的**Shape**引用来调用，也能产生正确行为。这就是多态。

因此，通常会创建一个具体对象（**Circle**，**Square**，或者**Triangle**），把它向上转型成**Shape**（忽略对象的具体类型），并在后面的程序中使用匿名（译注：即不知道具体类型）的Shape引用。

你可以像下面这样对Shape层次结构编码：

```
//: typeinfo/Shapes.java
import java.util.*;

abstract class Shape {
  void draw() { System.out.println(this + ".draw()"); }
  abstract public String toString();
}

class Circle extends Shape {
  public String toString() { return "Circle"; }
}

class Square extends Shape {
  public String toString() { return "Square"; }
}

class Triangle extends Shape {
  public String toString() { return "Triangle"; }
}
```

```
public class Shapes {
  public static void main(String[] args) {
    List<Shape> shapeList = Arrays.asList(
      new Circle(), new Square(), new Triangle()
    );
    for(Shape shape : shapeList)
      shape.draw();
  }
} /* Output:
Circle.draw()
Square.draw()
Triangle.draw()
*///:~
```

554

基类中包含**draw()**方法，它通过传递**this**参数给**System.out.println()**，间接地使用**toString()**打印类标识符（注意，toString()被声明为abstract，以此强制继承者覆写该方法，并可以防止对无格式的Shape的实例化）。如果某个对象出现在字符串表达式中（涉及“+”和字符串对象的表达式），toString()方法就会被自动调用，以生成表示该对象的String。每个派生类都要覆盖（从**Object**继承来的）**toString()**方法，这样**draw()**在不同情况下就打印出不同的消息（多态）。

在这个例子中，当把**Shape**对象放入**List<Shape>**的数组时会向上转型。但在向上转型为**Shape**的时候也丢失了**Shape**对象的具体类型。对于数组而言，它们只是**Shape**类的对象。

当从数组中取出元素时，这种容器——实际上它将所有的事物都当作Object持有——会自动将结果转型回Shape。这是RTTI最基本的使用形式，因为在Java中，所有的类型转换都是在运行时进行正确性检查的。这也是RTTI名字的含义：在运行时，识别一个对象的类型。

在这个例子中，RTTI类型转换并不彻底：**Object**被转型为**Shape**，而不是转型为**Circle**、**Square**或者**Triangle**。这是因为目前我们只知道这个**List<Shape>**保存的都是**Shape**。在编译时，将由容器和Java的泛型系统来强制确保这一点；而在运行时，由类型转换操作来确保这一点。

接下来就是多态机制的事情了，**Shape**对象实际执行什么样的代码，是由引用所指向的具体对象**Circle**、**Square**或者**Triangle**而决定的。通常，也正是这样要求的；你希望大部分代码尽可能少地了解对象的具体类型，而是只与对象家族中的一个通用表示打交道（在这个例子中是**Shape**）。这样代码会更容易写，更容易读，且更便于维护；设计也更容易实现、理解和改变。所以“多态”是面向对象编程的基本目标。

但是，假如你碰到了一个特殊的编程问题——如果能够知道某个泛化引用的确切类型，就可以使用最简单的方式去解决它，那么此时该怎么办呢？例如，假设我们允许用户将某一具体类型的几何形状全都变成某种特殊的颜色，以突出显示它们。通过这种方法，用户就能找出屏幕上所有被突出显示的三角形。或者，可能要用某个方法来旋转列出的所有图形，但想跳过圆形，因为对圆形进行旋转没有意义。使用RTTI，可以查询某个**Shape**引用所指向的对象的确切类型，然后选择或者剔除特例。

555

14.2 Class对象

要理解RTTI在Java中的工作原理，首先必须知道类型信息在运行时是如何表示的。这项工作是由称为Class对象的特殊对象完成的，它包含了与类有关的信息。事实上，**Class**对象就是用来创建类的所有的“常规”对象的。Java使用Class对象来执行其RTTI，即使你正在执行的是类似转型这样的操作。**Class**类还拥有大量的使用RTTI的其他方式。

类是程序的一部分，每个类都有一个**Class**对象。换言之，每当编写并且编译了一个新类，就会产生一个**Class**对象（更恰当地说，是被保存在一个同名的**.class**文件中）。为了生成这个类

的对象，运行这个程序的Java虚拟机（JVM）将使用被称为“类加载器”的子系统。

类加载器子系统实际上可以包含一条类加载器链，但是只有一个*原生类加载器*，它是JVM实现的一部分。原生类加载器加载的是所谓的*可信类*，包括Java API类，它们通常是从本地盘加载的。在这条链中，通常不需要添加额外的类加载器，但是如果你有特殊需求（例如以某种特殊的方式加载类，以支持Web服务器应用，或者在网络中下载类），那么你有一种方式可以挂接额外的类加载器。

所有的类都是在对其第一次使用时，动态加载到JVM中的。当程序创建第一个对类的静态成员的引用时，就会加载这个类。这个证明构造器也是类的静态方法，即使在构造器之前并没有使用**static**关键字。因此，使用**new**操作符创建类的新对象也会被当作对类的静态成员的引用。

因此，Java程序在它开始运行之前并非被完全加载，其各个部分是在必需时才加载的。这一点与许多传统语言都不同。动态加载使能的行为，在诸如C++这样的静态加载语言中是很难或者根本不可能复制的。 556

类加载器首先检查这个类的**Class**对象是否已经加载。如果尚未加载，默认的类加载器就会根据类名查找**.class**文件（例如，某个附加类加载器可能会在数据库中查找字节码）。在这个类的字节码被加载时，它们会接受验证，以确保其没有被破坏，并且不包含不良Java代码（这是Java中用于安全防范目的的措施之一）。

一旦某个类的**Class**对象被载入内存，它就被用来创建这个类的所有对象。下面的示范程序可以证明这一点：

```
//: typeinfo/SweetShop.java
// Examination of the way the class loader works.
import static net.mindview.util.Print.*;

class Candy {
  static { print("Loading Candy"); }
}

class Gum {
  static { print("Loading Gum"); }
}

class Cookie {
  static { print("Loading Cookie"); }
}

public class SweetShop {
  public static void main(String[] args) {
    print("inside main");
    new Candy();
    print("After creating Candy");
    try {
      Class.forName("Gum");
    } catch(ClassNotFoundException e) {
      print("Couldn't find Gum");
    }
    print("After Class.forName(\"Gum\")");
    new Cookie();
    print("After creating Cookie");
  }
} /* Output:
inside main
Loading Candy
After creating Candy
Loading Gum
After Class.forName("Gum")
Loading Cookie
```

557

```
After creating Cookie
*///:~
```

这里的每个类**Candy**、**Gum**和**Cookie**，都有一个**static**子句，该子句在类第一次被加载时执行。这时会有相应的信息打印出来，告诉我们这个类什么时候被加载了。在**main()**中，创建对象的代码被置于打印语句之间，以帮助我们判断加载的时间点。

从输出中可以看到，**Class**对象仅在需要的时候才被加载，**static**初始化是在类加载时进行的。

特别有趣的一行是：

```
Class.forName("Gum");
```

这个方法是**Class**类（所有Class对象都属于这个类）的一个**static**成员。**Class**对象就和其他对象一样，我们可以获取并操作它的引用（这也就是类加载器的工作）。**forName()**是取得**Class**对象的引用的一种方法。它是用一个包含目标类的文本名（注意拼写和大小写）的**String**作输入参数，返回的是一个**Class**对象的引用，上面的代码忽略了返回值。对**forName()**的调用是为了它产生的“副作用”：如果类**Gum**还没有被加载就加载它。在加载的过程中，**Gum**的**static**子句被执行。

在前面的例子里，如果**Class.forName()**找不到你要加载的类，它会抛出异常**ClassNotFoundException**。这里我们只需简单报告问题，但在更严密的程序里，可能要在异常处理程序中解决这个问题。

无论何时，只要你想在运行时使用类型信息，就必须首先获得对恰当的**Class**对象的引用。**Class.forName()**就是实现此功能的便捷途径，因为你不需要为了获得**Class**引用而持有该类型的对象。但是，如果你已经拥有了一个感兴趣的类型的对象，那就可以通过调用**getClass()**方法来获取**Class**引用了，这个方法属于根类**Object**的一部分，它将返回表示该对象的实际类型的**Class**引用。**Class**包含很多有用的方法，下面是其中的一部分：

558

```
//: typeinfo/toys/ToyTest.java
// Testing class Class.
package typeinfo.toys;
import static net.mindview.util.Print.*;

interface HasBatteries {}
interface Waterproof {}
interface Shoots {}

class Toy {
  // Comment out the following default constructor
  // to see NoSuchMethodError from (*1*)
  Toy() {}
  Toy(int i) {}
}

class FancyToy extends Toy
implements HasBatteries, Waterproof, Shoots {
  FancyToy() { super(1); }
}

public class ToyTest {
  static void printInfo(Class cc) {
    print("Class name: " + cc.getName() +
      " is interface? [" + cc.isInterface() + "]");
    print("Simple name: " + cc.getSimpleName());
    print("Canonical name : " + cc.getCanonicalName());
  }
  public static void main(String[] args) {
    Class c = null;
```

```
  public static void main(String[] args) {
    Class c = null;
    try {
      c = Class.forName("typeinfo.toys.FancyToy");
    } catch(ClassNotFoundException e) {
      print("Can't find FancyToy");
      System.exit(1);
    }
    printInfo(c);
    for(Class face : c.getInterfaces())
      printInfo(face);
    Class up = c.getSuperclass();
    Object obj = null;
    try {
      // Requires default constructor:
      obj = up.newInstance();
    } catch(InstantiationException e) {
      print("Cannot instantiate");
      System.exit(1);
    } catch(IllegalAccessException e) {
      print("Cannot access");
      System.exit(1);
    }
    printInfo(obj.getClass());
  }
} /* Output:
Class name: typeinfo.toys.FancyToy is interface? [false]
Simple name: FancyToy
Canonical name : typeinfo.toys.FancyToy
Class name: typeinfo.toys.HasBatteries is interface? [true]
Simple name: HasBatteries
Canonical name : typeinfo.toys.HasBatteries
Class name: typeinfo.toys.Waterproof is interface? [true]
Simple name: Waterproof
Canonical name : typeinfo.toys.Waterproof
Class name: typeinfo.toys.Shoots is interface? [true]
Simple name: Shoots
Canonical name : typeinfo.toys.Shoots
Class name: typeinfo.toys.Toy is interface? [false]
Simple name: Toy
Canonical name : typeinfo.toys.Toy
*///:~
```

559

FancyToy继承自**Toy**并实现了**HasBatteries**、**Waterproof**和**Shoots**接口。在**main()**中，用**forName()**方法在适当的**try**语句块中，创建了一个**Class**引用，并将其初始化为指向**FancyToy Class**。注意，在传递给**forName()**的字符串中，你必须使用全限定名（包含包名）。

printInfo()使用**getName()**来产生全限定的类名，并分别使用**getSimpleName()**和**getCanonicalName()**（在Java SE5中引入的）来产生不含包名的类名和全限定的类名。**isInterface()**方法如同其名，可以告诉你这个**Class**对象是否表示某个接口。因此，通过**Class**对象，你可以发现你想要了解的类型的所有信息。

在**main()**中调用的**Class.getInterfaces()**方法返回的是**Class**对象，它们表示在感兴趣的**Class**对象中所包含的接口。

560

如果你有一个**Class**对象，还可以使用**getSuperclass()**方法查询其直接基类，这将返回你可以用来进一步查询的**Class**对象。因此，你可以在运行时发现一个对象完整的类继承结构。

Class的**newInstance()**方法是实现“虚拟构造器”的一种途径，虚拟构造器允许你声明：“我不知道你的确切类型，但是无论如何要正确地创建你自己。”在前面的示例中，**up**仅仅只是一个**Class**引用，在编译期不具备任何更进一步的类型信息。当你创建新实例时，会得到**Object**引用，但是这个引用指向的是**Toy**对象。当然，在你可以发送**Object**能够接受的消息之外的任何

消息之前，你必须更多地了解它，并执行某种转型。另外，使用**newInstance()**来创建的类，必须带有默认的构造器。在本章稍后部分，你将会看到如何通过使用Java的反射API，用任意的构造器来动态地创建类的对象。

练习1：(1) 在**ToyTest.java**中，将**Toy**的默认构造器注释掉，并解释发生的现象。

练习2：(2) 将新的**interface**加到**ToyTest.java**中，看看这个程序是否能够正确检测出来并加以显示。

练习3：(2) 将**Rhomboid**（菱形）加入**Shapes.java**中。创建一个**Rhomboid**，将其向上转型为**Shape**，然后向下转型回**Rhomboid**。试着将其向下转型成**Circle**，看看会发生什么。

练习4：(2) 修改前一个练习，让你的程序在执行向下转型之前先运用**instanceof**检查类型。

练习5：(3) 实现**Shapes.java**中的**rotate(Shape)**方法，让它能判断它所旋转的是不是**Circle**（如果是，就不执行）。

练习6：(4) 修改**Shapes.java**，使这个程序能将某个特定类型的所有形状都“标示”出来（通过设标志）。每一个导出的**Shape**类的**toString()**方法应该更够指出**Shape**是否被标示。

练习7：(3)修改**SweetShop.java**，使每种类型对象的创建由命令行参数控制。即，如果命令行是“**java SweetShop Candy**”，那么只有**Candy**对象被创建。注意你是如何通过命令行参数来控制加载哪个**Class**对象的。

561

练习8：(5) 写一个方法，令它接受任意对象作为参数，并能够递归打印出该对象所在的继承体系中的所有类。

练习9：(5) 修改前一个练习，让这个方法使用**Class.getDeclaredFields()**来打印一个类中的域的相关信息。

练习10：(3) 写一个程序，使它能判断**char**数组究竟是个基本类型，还是一个对象。

14.2.1 类字面常量

Java 还提供了另一种方法来生成对**Class**对象的引用，即使用*类字面常量*。对上述程序来说，就像下面这样：

```
FancyToy.class;
```

这样做不仅更简单，而且更安全，因为它在编译时就会受到检查（因此不需要置于**try**语句块中）。并且它根除了对**forName()**方法的调用，所以也更高效。

类字面常量不仅可以应用于普通的类，也可以应用于接口、数组以及基本数据类型。另外，对于基本数据类型的包装器类，还有一个标准字段TYPE。TYPE字段是一个引用，指向对应的基本数据类型的Class对象，如下所示：

…等价于…	
boolean.class	**Boolean.TYPE**
char.class	**Character.TYPE**
byte.class	**Byte.TYPE**
short.class	**Short.TYPE**
int.class	**Integer.TYPE**
long.class	**Long.TYPE**
float.class	**Float.TYPE**
double.class	**Double.TYPE**
void.class	**Void.TYPE**

562

我建议使用“**.class**”的形式，以保持与普通类的一致性。

注意，有一点很有趣，当使用“**.class**”来创建对**Class**对象的引用时，不会自动地初始化该**Class**对象。为了使用类而做的准备工作实际包含三个步骤：

1. *加载*，这是由类加载器执行的。该步骤将查找字节码（通常在classpath所指定的路径中查找，但这并非是必需的），并从这些字节码中创建一个**Class**对象。

2. *链接*。在链接阶段将验证类中的字节码，为静态域分配存储空间，并且如果必需的话，将解析这个类创建的对其他类的所有引用。

3. *初始化*。如果该类具有超类，则对其初始化，执行静态初始化器和静态初始化块。

初始化被延迟到了对静态方法（构造器隐式地是静态的）或者非常数静态域进行首次引用时才执行：

```
//: typeinfo/ClassInitialization.java
import java.util.*;

class Initable {
  static final int staticFinal = 47;
  static final int staticFinal2 =
    ClassInitialization.rand.nextInt(1000);
  static {
    System.out.println("Initializing Initable");
  }
}

class Initable2 {
  static int staticNonFinal = 147;
  static {
    System.out.println("Initializing Initable2");
  }
}

class Initable3 {
  static int staticNonFinal = 74;
  static {
    System.out.println("Initializing Initable3");
  }
}

public class ClassInitialization {
  public static Random rand = new Random(47);
  public static void main(String[] args) throws Exception {
    Class initable = Initable.class;
    System.out.println("After creating Initable ref");
    // Does not trigger initialization:
    System.out.println(Initable.staticFinal);
    // Does trigger initialization:
    System.out.println(Initable.staticFinal2);
    // Does trigger initialization:
    System.out.println(Initable2.staticNonFinal);
    Class initable3 = Class.forName("Initable3");
    System.out.println("After creating Initable3 ref");
    System.out.println(Initable3.staticNonFinal);
  }
} /* Output:
After creating Initable ref
47
Initializing Initable
258
Initializing Initable2
147
Initializing Initable3
After creating Initable3 ref
```

563

```
74
*///:~
```

初始化有效地实现了尽可能的“惰性”。从对**initable**引用的创建中可以看到，仅使用**.class**语法来获得对类的引用不会引发初始化。但是，为了产生**Class**引用，**Class.forName()**立即就进行了初始化，就像在对**initable3**引用的创建中所看到的。

如果一个**static final**值是“编译期常量”，就像**Initable.staticFinal**那样，那么这个值不需要对**Initable**类进行初始化就可以被读取。但是，如果只是将一个域设置为**static**和**final**的，还不足以确保这种行为，例如，对**Initable.staticFinal2**的访问将强制进行类的初始化，因为它不是一个编译期常量。

564

如果一个**static**域不是**final**的，那么在对它访问时，总是要求在它被读取之前，要先进行链接（为这个域分配存储空间）和初始化（初始化该存储空间），就像在对**Initable2.staticNonFinal**的访问中所看到的那样。

14.2.2 泛化的Class引用

Class引用总是指向某个**Class**对象，它可以制造类的实例，并包含可作用于这些实例的所有方法代码。它还包含该类的静态成员，因此，**Class**引用表示的就是它所指向的对象的确切类型，而该对象便是**Class**类的一个对象。

但是，Java SE5的设计者们看准机会，将它的类型变得更具体了一些，而这是通过允许你对**Class**引用所指向的**Class**对象的类型进行限定而实现的，这里用到了泛型语法。在下面的实例中，两种语法都是正确的：

```
//: typeinfo/GenericClassReferences.java

public class GenericClassReferences {
  public static void main(String[] args) {
    Class intClass = int.class;
    Class<Integer> genericIntClass = int.class;
    genericIntClass = Integer.class; // Same thing
    intClass = double.class;
    // genericIntClass = double.class; // Illegal
  }
} ///:~
```

普通的类引用不会产生警告信息，你可以看到，尽管泛型类引用只能赋值为指向其声明的类型，但是普通的类引用可以被重新赋值为指向任何其他的**Class**对象。通过使用泛型语法，可以让编译器强制执行额外的类型检查。

如果你希望稍微放松一些这种限制，应该怎么办呢？乍一看，好像你应该能够执行类似下面这样的操作：

```
Class<Number> genericNumberClass = int.class;
```

这看起来似乎是起作用的，因为**Integer**继承自**Number**。但是它无法工作，因为**Integer Class**对象不是**Number Class**对象的子类（这种差异看起来可能有些诡异，我们将在第15章中深入讨论它）。

565

为了在使用泛化的**Class**引用时放松限制，我使用了通配符，它是Java泛型的一部分。通配符就是“**?**”，表示“任何事物”。因此，我们可以在上例的普通**Class**引用中添加通配符，并产生相同的结果：

```
//: typeinfo/WildcardClassReferences.java

public class WildcardClassReferences {
  public static void main(String[] args) {
```

```
    Class<?> intClass = int.class;
    intClass = double.class;
  }
} ///:~
```

在Java SE5中，**Class<?>**优于平凡的**Class**，即便它们是等价的，并且平凡的**Class**如你所见，不会产生编译器警告信息。**Class<?>**的好处是它表示你并非是碰巧或者由于疏忽，而使用了一个非具体的类引用，你就是选择了非具体的版本。

为了创建一个**Class**引用，它被限定为某种类型，或该类型的任何子类型，你需要将通配符与**extends**关键字相结合，创建一个范围。因此，与仅仅声明**Class<Number>**不同，现在做如下声明：

```
//: typeinfo/BoundedClassReferences.java

public class BoundedClassReferences {
  public static void main(String[] args) {
    Class<? extends Number> bounded = int.class;
    bounded = double.class;
    bounded = Number.class;
    // Or anything else derived from Number.
  }
} ///:~
```

向**Class**引用添加泛型语法的原因仅仅是为了提供编译期类型检查，因此如果你操作有误，稍后立即就会发现这一点。在使用普通**Class**引用，你不会误入歧途，但是如果你确实犯了错误，那么直到运行时你才会发现它，而这显得很不方便。

下面的示例使用了泛型类语法。它存储了一个类引用，稍候又产生了一个**List**，填充这个**List**的对象是使用**newInstance()**方法，通过该引用生成的：

566

```
//: typeinfo/FilledList.java
import java.util.*;

class CountedInteger {
  private static long counter;
  private final long id = counter++;
  public String toString() { return Long.toString(id); }
}

public class FilledList<T> {
  private Class<T> type;
  public FilledList(Class<T> type) { this.type = type; }
  public List<T> create(int nElements) {
    List<T> result = new ArrayList<T>();
    try {
      for(int i = 0; i < nElements; i++)
        result.add(type.newInstance());
    } catch(Exception e) {
      throw new RuntimeException(e);
    }
    return result;
  }
  public static void main(String[] args) {
    FilledList<CountedInteger> fl =
      new FilledList<CountedInteger>(CountedInteger.class);
    System.out.println(fl.create(15));
  }
} /* Output:
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14]
*///:~
```

注意，这个类必须假设与它一同工作的任何类型都具有一个默认的构造器（无参构造器），

并且如果不符合该条件，你将得到一个异常。编译器对该程序不会产生任何警告信息。

当你将泛型语法用于Class对象是，会发生一件很有趣的事情：newInstance()将返回该对象的确切类型，而不仅仅只是在ToyTest.java中看到的基本的Object。这在某种程度上有些受限：

567

```
//: typeinfo/toys/GenericToyTest.java
// Testing class Class.
package typeinfo.toys;
public class GenericToyTest {
  public static void main(String[] args) throws Exception {
    Class<FancyToy> ftClass = FancyToy.class;
    // Produces exact type:
    FancyToy fancyToy = ftClass.newInstance();
    Class<? super FancyToy> up = ftClass.getSuperclass();
    // This won't compile:
    // Class<Toy> up2 = ftClass.getSuperclass();
    // Only produces Object:
    Object obj = up.newInstance();
  }
} ///:~
```

如果你手头的是超类，那编译器将只允许你声明超类引用是“某个类，它是FancyToy超类”，就像在表达式Class<? Super FancyToy>中所看到的，而不会接受Class<Toy>这样的声明。这看上去显得有些怪，因为getSuperClass()方法返回的是基类（不是接口），并且编译器在编译期就知道它是什么类型了——在本例中就是Toy.class——而不仅仅只是“某个类，它是FancyToy超类”。不管怎样，正是由于这种含糊性，up.newInstance()的返回值不是精确类型，而只是Object。

14.2.3 新的转型语法

Java SE5 还添加了用于Class引用的转型语法，即cast()方法：

```
//: typeinfo/ClassCasts.java

class Building {}
class House extends Building {}

public class ClassCasts {
  public static void main(String[] args) {
    Building b = new House();
    Class<House> houseType = House.class;
    House h = houseType.cast(b);
    h = (House)b; // ... or just do this.
  }
} ///:~
```

cast()方法接受参数对象，并将其转型为Class引用的类型。当然，如果你观察上面的代码，则会发现，与实现了相同功能的main()中最后一行相比，这种转型好像做了很多额外的工作。新的转型语法对于无法使用普通转型的情况显得非常有用，在你编写泛型代码（你将在第15章中学习它）时，如果你存储了Class引用，并希望以后通过这个引用来执行转型，这种情况就会时有发生。这被证明是一种罕见的情况——我发现在整个Java SE5类库中，只有一处使用了cast()（在com.sun.mirror.util.DeclarationFilter中）。

568

在Java SE5中另一个没有任何用处的新特性就是Class.asSubclass()，该方法允许你将一个类对象转型为更加具体的类型。

14.3 类型转换前先做检查

迄今为止，我们已知的RTTI形式包括：

1) 传统的类型转换，如“(Shape)”，由RTTI确保类型转换的正确性，如果执行了一个错误

的类型转换，就会抛出一个**ClassCastException**异常。

2) 代表对象的类型的**Class**对象。通过查询**Class**对象可以获取运行时所需的信息。

在C++中，经典的类型转换“**(Shape)**”并不使用RTTI。它只是简单地告诉编译器将这个对象作为新的类型对待。而Java要执行类型检查，这通常被称为“类型安全的向下转型”。之所以叫“向下转型”，是由于类层次结构图从来就是这么排列的。如果将**Circle**类型转换为**Shape**类型被称作向上转型，那么将**Shape**转型为**Circle**，就被称为向下转型。但是，由于知道**Circle**肯定是一个**Shape**，所以编译器允许自由地做向上转型的赋值操作，而不需要任何显式的转型操作。编译器无法知道对于给定的Shape到底是什么Shape——它可能就是Shape，或者是Shape的子类型，例如Circle、Square、Triangle或某种其他的类型。在编译期。编译器只能知道它是Shape。因此，如果不使用显式的类型转换，编译器就不允许你执行向下转型赋值，以告知编译器你拥有额外的信息，这些信息使你知道该类型是某种特定类型（编译器将检查向下转型是否合理，因此它不允许向下转型到实际上不是待转型类的子类的类型上）。

RTTI在Java中还有第三种形式，就是关键字**instanceof**。它返回一个布尔值，告诉我们对象是不是某个特定类型的实例。可以用提问的方式使用它，就像这样：

569

```
if(x instanceof Dog)
  ((Dog)x).bark();
```

在将**x**转型成一个**Dog**前，上面的**if**语句会检查对象**x**是否从属于**Dog**类。进行向下转型前，如果没有其他信息可以告诉你这个对象是什么类型，那么使用**instanceof**是非常重要的，否则会得到一个**ClassCastException**异常。

一般，可能想要查找某种类型（比如要找三角形，并填充成紫色），这时可以轻松地使用**instanceof**来计数所有对象。例如，假设你有一个类的继承体系，描述了**Pet**（以及它们的主人，这是在后面的示例中出现的一个非常方便的特性）。在这个继承体系中的每个**Individual**都有一个**id**和一个可选的名字。尽管下面的类都继承自**Individual**，但是**Individual**类复杂性较高，因此其代码将放到第17章中进行说明与解释。正如你可以看到的，此处并不需要去了解**Individual**的代码——你只需了解你可以创建其具名或不具名的对象，并且每个**Individual**都有一个**id()**方法，可以返回其唯一的标识符（通过对每个对象计数而创建的）。还有一个**toString()**方法，如果你没有为**Individual**提供名字，**toString()**方法只产生类型名。

下面是继承自**Individual**的类继承体系：

```
//: typeinfo/pets/Person.java
package typeinfo.pets;

public class Person extends Individual {
  public Person(String name) { super(name); }
} ///:~

//: typeinfo/pets/Pet.java
package typeinfo.pets;

public class Pet extends Individual {
  public Pet(String name) { super(name); }
  public Pet() { super(); }
} ///:~

//: typeinfo/pets/Dog.java
package typeinfo.pets;

public class Dog extends Pet {
  public Dog(String name) { super(name); }
  public Dog() { super(); }
} ///:~
```

570

```
//: typeinfo/pets/Mutt.java
package typeinfo.pets;

public class Mutt extends Dog {
  public Mutt(String name) { super(name); }
  public Mutt() { super(); }
} ///:~

//: typeinfo/pets/Pug.java
package typeinfo.pets;

public class Pug extends Dog {
  public Pug(String name) { super(name); }
  public Pug() { super(); }
} ///:~

//: typeinfo/pets/Cat.java
package typeinfo.pets;

public class Cat extends Pet {
  public Cat(String name) { super(name); }
  public Cat() { super(); }
} ///:~

//: typeinfo/pets/EgyptianMau.java
package typeinfo.pets;

public class EgyptianMau extends Cat {
  public EgyptianMau(String name) { super(name); }
  public EgyptianMau() { super(); }
} ///:~

//: typeinfo/pets/Manx.java
package typeinfo.pets;

public class Manx extends Cat {
  public Manx(String name) { super(name); }
  public Manx() { super(); }
} ///:~

//: typeinfo/pets/Cymric.java
package typeinfo.pets;

public class Cymric extends Manx {
  public Cymric(String name) { super(name); }
  public Cymric() { super(); }
} ///:~

//: typeinfo/pets/Rodent.java
package typeinfo.pets;

public class Rodent extends Pet {
  public Rodent(String name) { super(name); }
  public Rodent() { super(); }
} ///:~

//: typeinfo/pets/Rat.java
package typeinfo.pets;

public class Rat extends Rodent {
  public Rat(String name) { super(name); }
  public Rat() { super(); }
} ///:~

//: typeinfo/pets/Mouse.java
package typeinfo.pets;

public class Mouse extends Rodent {
  public Mouse(String name) { super(name); }
  public Mouse() { super(); }
```

571

```
} ///:~
```

```
//: typeinfo/pets/Hamster.java
package typeinfo.pets;

public class Hamster extends Rodent {
  public Hamster(String name) { super(name); }
  public Hamster() { super(); }
} ///:~
```

接下来，我们需要一种方法，通过它可以随机地创建不同类型的宠物，并且为方便起见，还可以创建宠物数组和**List**。为了使该工具能够适应多种不同的实现，我们将其定义为抽象类：

```
//: typeinfo/pets/PetCreator.java
// Creates random sequences of Pets.
package typeinfo.pets;
import java.util.*;

public abstract class PetCreator {
  private Random rand = new Random(47);
  // The List of the different types of Pet to create:
  public abstract List<Class<? extends Pet>> types();
  public Pet randomPet() { // Create one random Pet
    int n = rand.nextInt(types().size());
    try {
      return types().get(n).newInstance();
    } catch(InstantiationException e) {
      throw new RuntimeException(e);
    } catch(IllegalAccessException e) {
      throw new RuntimeException(e);
    }
  }
  public Pet[] createArray(int size) {
    Pet[] result = new Pet[size];
    for(int i = 0; i < size; i++)
      result[i] = randomPet();
    return result;
  }
  public ArrayList<Pet> arrayList(int size) {
    ArrayList<Pet> result = new ArrayList<Pet>();
    Collections.addAll(result, createArray(size));
    return result;
  }
} ///:~
```

572

抽象的**getTypes()**方法在导出类中实现，以获取由**Class**对象构成的**List**（这是模版方法设计模式的一种变体）。注意，其中类的类型被指定为“任何从**Pet**导出的类”，因此**newInstance()**不需要转型就可以产生**Pet**。**randomPet()**随机地产生**List**中的索引，并使用被选取的**Class**对象，通过**Class.newInstance()**来生成该类的新实例。**createArray()**方法使用**randomPet()**来填充数组，而**arrayList()**方法使用的则是**createArray()**。

在调用**newInstance()**时，可能会得到两种异常，在紧跟**try**语句块后面的**catch**子句中可以看到对它们的处理。异常的名字再次成为了一种对错误类型相对比较有用的解释（**IllegalAccessException**表示违反了Java安全机制，在本例中，表示默认构造器为**private**的情况）。

当你导出**PetCreator**的子类时，唯一所需提供的就是你希望使用**randomPet()**和其他方法来创建的宠物类型的**List**。**getTypes()**方法通常只返回对一个静态**List**的引用。下面是使用**forName()**的一个具体实现：

573

```
//: typeinfo/pets/ForNameCreator.java
package typeinfo.pets;
import java.util.*;
```

```
public class ForNameCreator extends PetCreator {
  private static List<Class<? extends Pet>> types =
    new ArrayList<Class<? extends Pet>>();
  // Types that you want to be randomly created:
  private static String[] typeNames = {
    "typeinfo.pets.Mutt",
    "typeinfo.pets.Pug",
    "typeinfo.pets.EgyptianMau",
    "typeinfo.pets.Manx",
    "typeinfo.pets.Cymric",
    "typeinfo.pets.Rat",
    "typeinfo.pets.Mouse",
    "typeinfo.pets.Hamster"
  };
  @SuppressWarnings("unchecked")
  private static void loader() {
    try {
      for(String name : typeNames)
        types.add(
          (Class<? extends Pet>)Class.forName(name));
    } catch(ClassNotFoundException e) {
      throw new RuntimeException(e);
    }
  }
  static { loader(); }
  public List<Class<? extends Pet>> types() {return types;}
} ///:~
```

loader()方法用**Class.forName()**创建了**Class**对象的**List**，这可能会产生**ClassNotFoundException**异常，这么做是有意义的，因为你传递给它的是一个在编译期无法验证的**String**。由于**Pet**对象在**typeinfo**包中，因此必须使用包名来引用这些类。

为了产生具有实际类型的**Class**对象的**List**，必须使用转型，这会产生编译期警告。**loader()**方法被单独定义，然后被置于一个静态初始化子句中，因为**@SuppressWarnings**注解不能直接置于静态初始化子句之上。

574

为了对**Pet**进行计数，我们需要一个能够跟踪各种不同类型的**Pet**的数量的工具。**Map**是此需求的首选，其中键是**Pet**类型名，而值是保存**Pet**数量的**Integer**。通过这种方式，你可以询问："有多少个**Hamster**对象？"我们可以使用**instanceof**来对**Pet**进行计数：

```
//: typeinfo/PetCount.java
// Using instanceof.
import typeinfo.pets.*;
import java.util.*;
import static net.mindview.util.Print.*;

public class PetCount {
  static class PetCounter extends HashMap<String,Integer> {
    public void count(String type) {
      Integer quantity = get(type);
      if(quantity == null)
        put(type, 1);
      else
        put(type, quantity + 1);
    }
  }
  public static void
  countPets(PetCreator creator) {
    PetCounter counter= new PetCounter();
    for(Pet pet : creator.createArray(20)) {
      // List each individual pet:
      printnb(pet.getClass().getSimpleName() + " ");
      if(pet instanceof Pet)
        counter.count("Pet");
```

```
      if(pet instanceof Dog)
        counter.count("Dog");
      if(pet instanceof Mutt)
        counter.count("Mutt");
      if(pet instanceof Pug)
        counter.count("Pug");
      if(pet instanceof Cat)
        counter.count("Cat");
      if(pet instanceof Manx)
        counter.count("EgyptianMau");
      if(pet instanceof Manx)
        counter.count("Manx");
      if(pet instanceof Manx)
        counter.count("Cymric");
      if(pet instanceof Rodent)
        counter.count("Rodent");
      if(pet instanceof Rat)
        counter.count("Rat");
      if(pet instanceof Mouse)
        counter.count("Mouse");
      if(pet instanceof Hamster)
        counter.count("Hamster");
    }
    // Show the counts:
    print();
    print(counter);
  }
  public static void main(String[] args) {
    countPets(new ForNameCreator());
  }
} /* Output:
Rat Manx Cymric Mutt Pug Cymric Pug Manx Cymric Rat
EgyptianMau Hamster EgyptianMau Mutt Mutt Cymric Mouse Pug
Mouse Cymric
{Pug=3, Cat=9, Hamster=1, Cymric=7, Mouse=2, Mutt=3,
Rodent=5, Pet=20, Manx=7, EgyptianMau=7, Dog=6, Rat=2}
*///:~
```

575

在**CountPets()**中，是使用**PetCreator**来随机地向数组中填充**Pet**的。然后使用**instanceof**对该数组中的每个**Pet**进行测试和计数。

对**instanceof**有比较严格的限制：只可将其与命名类型进行比较，而不能与**Class**对象作比较。在前面的例子中，可能觉得写出那么一大堆**instanceof**表达式是很乏味的，的确如此。但是也没有办法让**instanceof**聪明起来，让它能够自动地创建一个**Class**对象的数组，然后将目标对象与这个数组中的对象进行逐一的比较（稍后会看到一个替代方案）。其实这并非是一种如你想象中那般好的限制，因为渐渐地读者就会理解，如果程序中编写了许多的**instanceof**表达式，就说明你的设计可能存在瑕疵。

14.3.1 使用类字面常量

如果我们用类字面常量重新实现PetCount，那么改写后的结果在许多方面都会显得更加清晰：

576

```
//: typeinfo/pets/LiteralPetCreator.java
// Using class literals.
package typeinfo.pets;
import java.util.*;

public class LiteralPetCreator extends PetCreator {
  // No try block needed.
  @SuppressWarnings("unchecked")
  public static final List<Class<? extends Pet>> allTypes =
    Collections.unmodifiableList(Arrays.asList(
```

```
        Pet.class, Dog.class, Cat.class,  Rodent.class,
        Mutt.class, Pug.class, EgyptianMau.class, Manx.class,
        Cymric.class, Rat.class, Mouse.class,Hamster.class));
  // Types for random creation:
  private static final List<Class<? extends Pet>> types =
    allTypes.subList(allTypes.indexOf(Mutt.class),
      allTypes.size());
  public List<Class<? extends Pet>> types() {
    return types;
  }
  public static void main(String[] args) {
    System.out.println(types);
  }
} /* Output:
[class typeinfo.pets.Mutt, class typeinfo.pets.Pug, class
typeinfo.pets.EgyptianMau, class typeinfo.pets.Manx, class
typeinfo.pets.Cymric, class typeinfo.pets.Rat, class
typeinfo.pets.Mouse, class typeinfo.pets.Hamster]
*///:~
```

在即将出现的**PetCount3.java**示例中，我们需要先用所有的**Pet**类型来预加载一个**Map**（而仅仅只是那些将要随机生成的类型），因此**allTypes List**是必需的。**types**列表是**allTypes**的一部分（通过使用**List.subList()**创建的），它包含了确切的宠物类型，因此它被用于随机**Pet**生成。

这一次，生成**types**的代码不需要放在**try**块内，因为它会在编译时得到检查，因此，它不会抛出任何异常，这与**Class.forName()**不一样。

我们现在在**typeinfo.pets**类库中有了两种**PetCreator**的实现。为了将第二种实现作为默认实现，我们可以创建一个使用了**LiteralPetCreator**的外观：

577

```
//: typeinfo/pets/Pets.java
// Facade to produce a default PetCreator.
package typeinfo.pets;
import java.util.*;

public class Pets {
  public static final PetCreator creator =
    new LiteralPetCreator();
  public static Pet randomPet() {
    return creator.randomPet();
  }
  public static Pet[] createArray(int size) {
    return creator.createArray(size);
  }
  public static ArrayList<Pet> arrayList(int size) {
    return creator.arrayList(size);
  }
} ///:~
```

这个类还提供了对**randomPet()**、**createArray()**和**arrayList()**的间接调用。

因为**PetCount.countPets()**接受的是一个**PetCreator**参数，我们可以很容易地测试**LiteralPetCreator**（通过上面的外观）：

```
//: typeinfo/PetCount2.java
import typeinfo.pets.*;

public class PetCount2 {
  public static void main(String[] args) {
    PetCount.countPets(Pets.creator);
  }
} /* (Execute to see output) *///:~
```

该示例的输出与**PetCount.java**相同。

14.3.2 动态的instanceof

Class.isInstance方法提供了一种动态地测试对象的途径。于是所有那些单调的**instanceof**语句都可以从**PetCount.java**的例子中移除了。如下所示：

```
//: typeinfo/PetCount3.java
// Using isInstance()
import typeinfo.pets.*;
import java.util.*;
import net.mindview.util.*;
import static net.mindview.util.Print.*;

public class PetCount3 {
  static class PetCounter
  extends LinkedHashMap<Class<? extends Pet>,Integer> {
    public PetCounter() {
      super(MapData.map(LiteralPetCreator.allTypes, 0));
    }
    public void count(Pet pet) {
      // Class.isInstance() eliminates instanceofs:
      for(Map.Entry<Class<? extends Pet>,Integer> pair
          : entrySet())
        if(pair.getKey().isInstance(pet))
          put(pair.getKey(), pair.getValue() + 1);
    }
    public String toString() {
      StringBuilder result = new StringBuilder("{");
      for(Map.Entry<Class<? extends Pet>,Integer> pair
          : entrySet()) {
        result.append(pair.getKey().getSimpleName());
        result.append("=");
        result.append(pair.getValue());
        result.append(", ");
      }
      result.delete(result.length()-2, result.length());
      result.append("}");
      return result.toString();
    }
  }
  public static void main(String[] args) {
    PetCounter petCount = new PetCounter();
    for(Pet pet : Pets.createArray(20)) {
      printnb(pet.getClass().getSimpleName() + " ");
      petCount.count(pet);
    }
    print();
    print(petCount);
  }
} /* Output:
Rat Manx Cymric Mutt Pug Cymric Pug Manx Cymric Rat
EgyptianMau Hamster EgyptianMau Mutt Mutt Cymric Mouse Pug
Mouse Cymric
{Pet=20, Dog=6, Cat=9, Rodent=5, Mutt=3, Pug=3,
EgyptianMau=2, Manx=7, Cymric=5, Rat=2, Mouse=2, Hamster=1}
*///:~
```

578

579

为了对所有不同类型的**Pet**进行计数，**PetCounter Map**预加载了**LiteralPetCreator.allTypes**中的类型。这使用了**net.mindview.util.MapData**类，这个类接受一个**Iteralbe**（**allTypes List**）和一个常数值（在本例中是0），然后用**allTypes**中元素作为键，用0作为值，来填充**Map**。如果不预加载这个**Map**，那么你最终将只能对随机生成的类型进行计数，而不包括诸如**Pet**和**Cat**这样的基类型。

可以看到，**isInstance()**方法使我们不再需要**instanceof**表达式。此外，这意味着如果要求添加新类型的**Pet**，只需简单地改变**LiteralPetCreator.java**数组即可；而毋需改动程序其他部分

(但是在使用**instanceof**时这是不可能的)。

toString()方法已经被重载，使得输出更容易被读取，而该输出与打印**Map**时所看到的典型输出仍然是匹配的。

14.3.3 递归计数

在**PetCount3.PetCounter**中的**Map**预加载了所有不同的**Pet**类。与预加载映射表不同的是，我们可以使用**Class.isAssignableFrom()**，并创建一个不局限于对**Pet**计数的通用工具。

```
//: net/mindview/util/TypeCounter.java
// Counts instances of a type family.
package net.mindview.util;
import java.util.*;

public class TypeCounter extends HashMap<Class<?>,Integer>{
  private Class<?> baseType;
  public TypeCounter(Class<?> baseType) {
    this.baseType = baseType;
  }
  public void count(Object obj) {
    Class<?> type = obj.getClass();
    if(!baseType.isAssignableFrom(type))
      throw new RuntimeException(obj + " incorrect type: "
        + type + ", should be type or subtype of "
        + baseType);
    countClass(type);
  }
  private void countClass(Class<?> type) {
    Integer quantity = get(type);
    put(type, quantity == null ? 1 : quantity + 1);
    Class<?> superClass = type.getSuperclass();
    if(superClass != null &&
        baseType.isAssignableFrom(superClass))
      countClass(superClass);
  }
  public String toString() {
    StringBuilder result = new StringBuilder("{");
    for(Map.Entry<Class<?>,Integer> pair : entrySet()) {
      result.append(pair.getKey().getSimpleName());
      result.append("=");
      result.append(pair.getValue());
      result.append(", ");
    }
    result.delete(result.length()-2, result.length());
    result.append("}");
    return result.toString();
  }
} ///:~
```

580

count()方法获取其参数的**Class**，然后使用**isAssignableFrom()**来执行运行时的检查，以校验你传递的对象确实属于我们感兴趣的继承结构。**countClass()**首先对该类的确切类型计数，然后，如果其超类可以赋值给**baseType**，**countClass()**将其超类上递归计数。

```
//: typeinfo/PetCount4.java
import typeinfo.pets.*;
import net.mindview.util.*;
import static net.mindview.util.Print.*;

public class PetCount4 {
  public static void main(String[] args) {
    TypeCounter counter = new TypeCounter(Pet.class);
    for(Pet pet : Pets.createArray(20)) {
      printnb(pet.getClass().getSimpleName() + " ");
      counter.count(pet);
    }
```

581

```
    print();
    print(counter);
  }
} /* Output: (Sample)
Rat Manx Cymric Mutt Pug Cymric Pug Manx Cymric Rat
EgyptianMau Hamster EgyptianMau Mutt Mutt Cymric Mouse Pug
Mouse Cymric
{Mouse=2, Dog=6, Manx=7, EgyptianMau=2, Rodent=5, Pug=3,
Mutt=3, Cymric=5, Cat=9, Hamster=1, Pet=20, Rat=2}
*///:~
```

正如在输出中看到的那样，对基类型和确切类型都进行了计数。

练习11：(2) 在**typeinfo.pets**类库中添加**Gerbil**，并修改本章中的所有示例，让它们适应这个新类。

练习12：(3) 将第15章中的**CoffeeGenerator.java**类用于**TypeCounter**。

练习13：(3) 将本章中的**RegisteredFactories.java**示例用于**TypeCounter**。

14.4 注册工厂

生成**Pet**继承结构中的对象存在着一个问题，即每次向该继承结构添加新的**Pet**类型时，必须将其添加为**LiteralPetCreator.java**中的项。如果在系统中已经存在了继承结构的常规的基础，然后在其上要添加更多的类，那么就有可能会出现问题。

你可能会考虑在每个子类中添加静态初始化器，以使得该初始化器可以将它的类添加到某个**List**中。遗憾的是，静态初始化器只有在类首先被加载的情况下才能被调用，因此你就碰上了“先有鸡还是先有蛋”的问题：生成器在其列表中不包含这个类，因此它永远不能创建这个类的对象，而这个类也就不能被加载并置于这个列表中。

这主要是因为，你被强制要求自己去手工创建这个列表（除非你想编写一个工具，它可以全面搜索和分析源代码，然后创建和编译这个列表）。因此，你最佳的做法是，将这个列表置于一个位于中心的、位置明显的地方，而我们感兴趣的继承结构的基类可能就是这个最佳位置。 582

这里我们需要做的其他修改就是使用*工厂方法*设计模式，将对象的创建工作交给类自己去完成。工厂方法可以被多态地调用，从而为你创建恰当类型的对象。在下面这个非常简单的版本中，工厂方法就是**Factory**接口中的**create()**方法：

```
//: typeinfo/factory/Factory.java
package typeinfo.factory;
public interface Factory<T> { T create(); } ///:~
```

泛型参数**T**使得**create()**可以在每种**Factory**实现中返回不同的类型。这也充分利用了协变返回类型。

在下面的示例中，基类**Part**包含一个工厂对象的列表。对于应这个由**createRandom()**方法产生的类型，它们的工厂都被添加到了**partFactories List**中，从而被注册到了基类中：

```
//: typeinfo/RegisteredFactories.java
// Registering Class Factories in the base class.
import typeinfo.factory.*;
import java.util.*;

class Part {
  public String toString() {
    return getClass().getSimpleName();
  }
  static List<Factory<? extends Part>> partFactories =
    new ArrayList<Factory<? extends Part>>();
  static {
```

```
    // Collections.addAll() gives an "unchecked generic
    // array creation ... for varargs parameter" warning.
    partFactories.add(new FuelFilter.Factory());
    partFactories.add(new AirFilter.Factory());
    partFactories.add(new CabinAirFilter.Factory());
    partFactories.add(new OilFilter.Factory());
    partFactories.add(new FanBelt.Factory());
    partFactories.add(new PowerSteeringBelt.Factory());
    partFactories.add(new GeneratorBelt.Factory());
  }
  private static Random rand = new Random(47);
  public static Part createRandom() {
    int n = rand.nextInt(partFactories.size());
    return partFactories.get(n).create();
  }
}

class Filter extends Part {}

class FuelFilter extends Filter {
  // Create a Class Factory for each specific type:
  public static class Factory
  implements typeinfo.factory.Factory<FuelFilter> {
    public FuelFilter create() { return new FuelFilter(); }
  }
}

class AirFilter extends Filter {
  public static class Factory
  implements typeinfo.factory.Factory<AirFilter> {
    public AirFilter create() { return new AirFilter(); }
  }
}

class CabinAirFilter extends Filter {
  public static class Factory
  implements typeinfo.factory.Factory<CabinAirFilter> {
    public CabinAirFilter create() {
      return new CabinAirFilter();
    }
  }
}

class OilFilter extends Filter {
  public static class Factory
  implements typeinfo.factory.Factory<OilFilter> {
    public OilFilter create() { return new OilFilter(); }
  }
}

class Belt extends Part {}

class FanBelt extends Belt {
  public static class Factory
  implements typeinfo.factory.Factory<FanBelt> {
    public FanBelt create() { return new FanBelt(); }
  }
}

class GeneratorBelt extends Belt {
  public static class Factory
  implements typeinfo.factory.Factory<GeneratorBelt> {
    public GeneratorBelt create() {
      return new GeneratorBelt();
    }
  }
}
```

583

584

```
class PowerSteeringBelt extends Belt {
  public static class Factory
  implements typeinfo.factory.Factory<PowerSteeringBelt> {
    public PowerSteeringBelt create() {
      return new PowerSteeringBelt();
    }
  }
}

public class RegisteredFactories {
  public static void main(String[] args) {
    for(int i = 0; i < 10; i++)
      System.out.println(Part.createRandom());
  }
} /* Output:
GeneratorBelt
CabinAirFilter
GeneratorBelt
AirFilter
PowerSteeringBelt
CabinAirFilter
FuelFilter
PowerSteeringBelt
PowerSteeringBelt
FuelFilter
*///:~
```

并非所有在继承结构中的类都应该被实例化，在本例中，**Filter**和**Belt**只是分类标识，因此你不应该创建它们的实例，而只应该创建它们的子类的实例。如果某个类应该由**createRandom()**方法创建，那么它就包含一个内部**Factory**类。如上所示，重用名字**Factory**的唯一方式就是限定**typeinfo.factory.Factory**。 585

尽管你可以使用**Collections.addAll()**来向列表中添加工厂，但是这样做编译器就会表达它的不满，抛出一条有关“创建泛型数组”（这被认为是不可能的，正如你将在第15章中所看到的那样）的警告，因此我转而使用**add()**。**createRandom()**方法从**partFactories**中随机地选取一个工厂对象，然后调用其**create()**方法，从而产生一个新的**Part**。

练习14：(4) 构造器就是一种工厂方法。修改**RegisteredFactories.java**，使其不要使用显式的工厂，而是将类对象存储到**List**中，并使用**newInstance()**来创建对象。

练习15：(4) 使用注册工厂来实现一个新的**PetCreator**，并修改**Pets**外观，使其使用这个新的**Creator**而并非另外两个**Creator**。确保使用**Pets.java**的其他示例仍可以正常工作。

练习16：(4) 修改第15章中的**Coffee**继承结构，以便可以使用注册工厂。

14.5 instanceof与Class的等价性

在查询类型信息时，以**instanceof**的形式（即以**instanceof**的形式或**isInstance()**的形式，它们产生相同的结果）与直接比较**Class**对象有一个很重要的差别。下面的例子展示了这种差别：

```
//: typeinfo/FamilyVsExactType.java
// The difference between instanceof and class
package typeinfo;
import static net.mindview.util.Print.*;

class Base {}
class Derived extends Base {}

public class FamilyVsExactType {
  static void test(Object x) {
    print("Testing x of type " + x.getClass());
    print("x instanceof Base " + (x instanceof Base));
```

586

```
    print("x instanceof Derived "+ (x instanceof Derived));
    print("Base.isInstance(x) "+ Base.class.isInstance(x));
    print("Derived.isInstance(x) "+
      Derived.class.isInstance(x));
    print("x.getClass() == Base.class " +
      (x.getClass() == Base.class));
    print("x.getClass() == Derived.class " +
      (x.getClass() == Derived.class));
    print("x.getClass().equals(Base.class)) "+
      (x.getClass().equals(Base.class)));
    print("x.getClass().equals(Derived.class)) " +
      (x.getClass().equals(Derived.class)));
  }
  public static void main(String[] args) {
    test(new Base());
    test(new Derived());
  }
} /* Output:
Testing x of type class typeinfo.Base
x instanceof Base true
x instanceof Derived false
Base.isInstance(x) true
Derived.isInstance(x) false
x.getClass() == Base.class true
x.getClass() == Derived.class false
x.getClass().equals(Base.class)) true
x.getClass().equals(Derived.class)) false
Testing x of type class typeinfo.Derived
x instanceof Base true
x instanceof Derived true
Base.isInstance(x) true
Derived.isInstance(x) true
x.getClass() == Base.class false
x.getClass() == Derived.class true
x.getClass().equals(Base.class)) false
x.getClass().equals(Derived.class)) true
*///:~
```

test()方法使用了两种形式的**instanceof**作为参数来执行类型检查。然后获取**Class**引用，并用**==**和**equals()**来检查**Class**对象是否相等。使人放心的是，**instancof**和**isInstance()**生成的结果完全一样，**equals()**和**==**也一样。但是这两组测试得出的结论却不相同。**instanceof**保持了类型的概念，它指的是“你是这个类吗，或者你是这个类的派生类吗？”而如果用**==**比较实际的**Class**对象，就没有考虑继承——它或者是这个确切的类型，或者不是。

587

14.6 反射：运行时的类信息

如果不知道某个对象的确切类型，RTTI可以告诉你。但是有一个限制：这个类型在编译时必须已知，这样才能使用RTTI识别它，并利用这些信息做一些有用的事。换句话说，在编译时，编译器必须知道所有要通过RTTI来处理的类。

初看起来这似乎不是个限制，但是假设你获取了一个指向某个并不在你的程序空间中的对象的引用；事实上，在编译时你的程序根本没法获知这个对象所属的类。例如，假设你从磁盘文件，或者网络连接中获取了一串字节，并且你被告知这些字节代表了一个类。既然这个类在编译器为你的程序生成代码之后很久才会出现，那么怎样才能使用这样的类呢？

在传统的编程环境中不太可能出现这种情况。但当我们置身于更大规模的编程世界中，在许多重要情况下就会发生上面的事情。首先就是“基于构件的编程”，在此种编程方式中，将使用某种基于*快速应用开发*（RAD）的应用构建工具，即*集成开发环境*（IDE），来构建项目。这是一种可视化编程方法，可以通过将代表不同组件的图标拖曳到表单中来创建程序。然后在编

程时通过设置构件的属性值来配置它们。这种设计时的配置，要求构件都是可实例化的，并且要暴露其部分信息，以允许程序员读取和修改构件的属性。此外，处理图形化用户界面（GUI）事件的构件还必须暴露相关方法的信息，以便IDE能够帮助程序员覆盖这些处理事件的方法。反射提供了一种机制——用来检查可用的方法，并返回方法名。Java通过JavaBeans（第22章将详细介绍）提供了基于构件的编程架构。

人们想要在运行时获取类的信息的另一个动机，便是希望提供在跨网络的远程平台上创建和运行对象的能力。这被称为*远程方法调用*（RMI），它允许一个Java程序将对象分布到多台机器上。需要这种分布能力是有许多原因的，例如，你可能正在执行一项需进行大量计算的任务，为了提高运算速度，想将计算划分为许多小的计算单元，分布到空闲的机器上运行。又比如，你可能希望将处理特定类型任务的代码（例如多层的C/S（客户/服务器）架构中的“业务规则”），置于特定的机器上，于是这台机器就成为了描述这些动作的公共场所，可以很容易地通过改动它就达到影响系统中所有人的效果。（这是一种有趣的开发方式，因为机器的存在仅仅是为了方便软件的改动！）同时，分布式计算也支持适于执行特殊任务的专用硬件，例如矩阵转置，而这对于通用程序就显得不太合适或者太昂贵了。

588

Class类与**java.lang.reflect**类库一起对反射的概念进行了支持，该类库包含了**Field**、**Method**以及**Constructor**类（每个类都实现了**Member**接口）。这些类型的对象是由JVM在运行时创建的，用以表示未知类里对应的成员。这样你就可以使用**Constructor**创建新的对象，用**get()**和**set()**方法读取和修改与**Field**对象关联的字段，用**invoke()**方法调用与**Method**对象关联的方法。另外，还可以调用**getFields()**、**getMethods()**和**getConstructors()**等很便利的方法，以返回表示字段、方法以及构造器的对象的数组（在JDK文档中，通过查找**Class**类可了解更多相关资料）。这样，匿名对象的类信息就能在运行时被完全确定下来，而在编译时不需要知道任何事情。

重要的是，要认识到反射机制并没有什么神奇之处。当通过反射与一个未知类型的对象打交道时，JVM只是简单地检查这个对象，看它属于哪个特定的类（就像RTTI那样）。在用它做其他事情之前必须先加载那个类的**Class**对象。因此，那个类的**.class**文件对于JVM来说必须是可获取的：要么在本地机器上，要么可以通过网络取得。所以RTTI和反射之间真正的区别只在于，对RTTI来说，编译器在编译时打开和检查**.class**文件。（换句话说，我们可以用“普通”方式调用对象的所有方法。）而对于反射机制来说，**.class**文件在编译时是不可获取的，所以是在运行时打开和检查**.class**文件。

589

14.6.1 类方法提取器

通常你不需要直接使用反射工具，但是它们在你需要创建更加动态的代码时会很有用。反射在Java中是用来支持其他特性的，例如对象序列化和JavaBean（它们在本书稍后部分都会提到）。但是，如果能动态地提取某个类的信息有的时候还是很有用的。请考虑类方法提取器。浏览实现了类定义的源代码或是其JDK文档，只能找到在这个类定义中被定义或被覆盖的方法。但对你来说，可能有数十个更有用的方法都是继承自基类的。要找出这些方法可能会很乏味且费时[⊖]。幸运的是，反射机制提供了一种方法，使我们能够编写可以自动展示完整接口的简单工具。下面就是其工作方式：

```
//: typeinfo/ShowMethods.java
// Using reflection to show all the methods of a class,
// even if the methods are defined in the base class.
```

⊖ 特别是在过去，现在Sun已极大地改进了其HTML Java 文档，所以查找基类的方法已经简单多了。

```
// {Args: ShowMethods}
import java.lang.reflect.*;
import java.util.regex.*;
import static net.mindview.util.Print.*;

public class ShowMethods {
  private static String usage =
    "usage:\n" +
    "ShowMethods qualified.class.name\n" +
    "To show all methods in class or:\n" +
    "ShowMethods qualified.class.name word\n" +
    "To search for methods involving 'word'";
  private static Pattern p = Pattern.compile("\\w+\\.");
  public static void main(String[] args) {
    if(args.length < 1) {
      print(usage);
      System.exit(0);
    }
    int lines = 0;
    try {
      Class<?> c = Class.forName(args[0]);
      Method[] methods = c.getMethods();
      Constructor[] ctors = c.getConstructors();
      if(args.length == 1) {
        for(Method method : methods)
          print(
            p.matcher(method.toString()).replaceAll(""));
        for(Constructor ctor : ctors)
          print(p.matcher(ctor.toString()).replaceAll(""));
        lines = methods.length + ctors.length;
      } else {
        for(Method method : methods)
          if(method.toString().indexOf(args[1]) != -1) {
            print(
              p.matcher(method.toString()).replaceAll(""));
            lines++;
          }
        for(Constructor ctor : ctors)
          if(ctor.toString().indexOf(args[1]) != -1) {
            print(p.matcher(
              ctor.toString()).replaceAll(""));
            lines++;
          }
      }
    } catch(ClassNotFoundException e) {
      print("No such class: " + e);
    }
  }
} /* Output:
public static void main(String[])
public native int hashCode()
public final native Class getClass()
public final void wait(long,int) throws
InterruptedException
public final void wait() throws InterruptedException
public final native void wait(long) throws
InterruptedException
public boolean equals(Object)
public String toString()
public final native void notify()
public final native void notifyAll()
public ShowMethods()
*///:~
```

590

591

Class的getMethods()和getConstructors()方法分别返回Method对象的数组和Constructor对象的数组。这两个类都提供了深层方法，用以解析其对象所代表的方法，并获取其名字、输

入参数以及返回值。但也可以像这里一样，只使用**toString()**生成一个含有完整的方法特征签名的字符串。代码其他部分用于提取命令行信息，判断某个特定的特征签名是否与我们的目标字符串相符（使用**indexOf()**），并使用正则表达式去掉了命名修饰词（正则表达式在第13章中介绍过）。

Class.forName()生成的结果在编译时是不可知的，因此所有的方法特征签名信息都是在执行时被提取出来的。如果研究一下JDK文档中关于反射的部分，就会看到，反射机制提供了足够的支持，使得能够创建一个在编译时完全未知的对象，并调用此对象的方法（在本书后面会有示例）。虽然开始的时候可能认为永远也不需要用到这些功能，但是反射机制的价值是很惊人的。

上面的输出是从下面的命令行产生的：

```
java ShowMethods ShowMethods
```

你可以看到，输出中包含一个**public**的默认构造器，即便能在代码中看到根本没有定义任何构造器。所看到的这个包含在列表中的构造器是编译器自动合成的。如果将**ShowMethods**作为一个非**public**的类（也就是拥有包访问权限），输出中就不会再显示出这个自动合成的默认构造器了。该自动合成的默认构造器会自动被赋予与类一样的访问权限。

还有一个有趣的例子是，用一个额外的**char**、**int**或**String**等参数来调用**java ShowMethods java.lang.String**。

在编程时，特别是如果不记得一个类是否有某个方法，或者不知道一个类究竟能做些什么，例如**Color**对象，而又不想通过索引或类的层次结构去查找JDK文档，这时这个工具确实能节省很多时间。

第22章包含这个程序的GUI版本（专为提取Swing构件的信息而定制的），使你在编写代码的同时能够通过运行它来快速查询有用的信息。 592

练习17：(2) 修改**ShowMethods.java**中的正则表达式，以去掉**native**和**final**关键字（提示：使用“或”运算符“|”）。

练习18：(1) 将**ShowMethods**变为一个非**public**的类，并验证合成的默认构造器不会再在输出中出现。

练习19：(4) 在**ToyTest.java**中，使用反射机制，通过非默认构造器创建**Toy**对象。

练习20：(5) 请从在http://java.sun.com上提供的JDK文档中找出**java.lang.Class**的接口。写一个程序，使它能够接受命令行参数所指定的类名称。然后使用**Class**的方法打印该类所有可以获得的信息。用标准程序库的类和你自己写的类，分别测试这个程序。

14.7 动态代理

*代理*是基本的设计模式之一，它是你为了提供额外的或不同的操作，而插入的用来代替“实际”对象的对象。这些操作通常涉及与“实际”对象的通信，因此代理通常充当着中间人的角色。下面是一个用来展示代理结构的简单示例：

```
//: typeinfo/SimpleProxyDemo.java
import static net.mindview.util.Print.*;

interface Interface {
  void doSomething();
  void somethingElse(String arg);
}

class RealObject implements Interface {
```

```
    public void doSomething() { print("doSomething"); }
    public void somethingElse(String arg) {
      print("somethingElse " + arg);
    }
  }

  class SimpleProxy implements Interface {
    private Interface proxied;
    public SimpleProxy(Interface proxied) {
      this.proxied = proxied;
    }
    public void doSomething() {
      print("SimpleProxy doSomething");
      proxied.doSomething();
    }
    public void somethingElse(String arg) {
      print("SimpleProxy somethingElse " + arg);
      proxied.somethingElse(arg);
    }
  }

  class SimpleProxyDemo {
    public static void consumer(Interface iface) {
      iface.doSomething();
      iface.somethingElse("bonobo");
    }
    public static void main(String[] args) {
      consumer(new RealObject());
      consumer(new SimpleProxy(new RealObject()));
    }
  } /* Output:
  doSomething
  somethingElse bonobo
  SimpleProxy doSomething
  doSomething
  SimpleProxy somethingElse bonobo
  somethingElse bonobo
  *///:~
```

593

因为**consumer()**接受的**Interface**，所以它无法知道正在获得的到底是**RealObject**还是**SimpleProxy**，因为这二者都实现了**Interface**。但是**SimpleProxy**已经被插入到了客户端和**RealObject**之间，因此它会执行操作，然后调用**RealObject**上相同的方法。

在任何时刻，只要你想要将额外的操作从“实际”对象中分离到不同的地方，特别是当你希望能够很容易地做出修改，从没有使用额外操作转为使用这些操作，或者反过来时，代理就显得很有用（设计模式的关键就是封装修改——因此你需要修改事务以证明这种模式的正确性）。例如，如果你希望跟踪对**RealObject**中的方法的调用，或者希望度量这些调用的开销，那么你应该怎样做呢？这些代码肯定是你不希望将其合并到应用中的代码，因此代理使得你可以很容易地添加或移除它们。

Java的*动态代理*比代理的思想更向前迈进了一步，因为它可以动态地创建代理并动态地处理对所代理方法的调用。在动态代理上所做的所有调用都会被重定向到单一的*调用处理器*上，它的工作是揭示调用的类型并确定相应的对策。下面是用动态代理重写的**SimpleProxyDemo.java**：

594

```
//: typeinfo/SimpleDynamicProxy.java
import java.lang.reflect.*;

class DynamicProxyHandler implements InvocationHandler {
  private Object proxied;
  public DynamicProxyHandler(Object proxied) {
    this.proxied = proxied;
```

```
  }
  public Object
  invoke(Object proxy, Method method, Object[] args)
  throws Throwable {
    System.out.println("**** proxy: " + proxy.getClass() +
      ", method: " + method + ", args: " + args);
    if(args != null)
      for(Object arg : args)
        System.out.println("  " + arg);
    return method.invoke(proxied, args);
  }
}

class SimpleDynamicProxy {
  public static void consumer(Interface iface) {
    iface.doSomething();
    iface.somethingElse("bonobo");
  }
  public static void main(String[] args) {
    RealObject real = new RealObject();
    consumer(real);
    // Insert a proxy and call again:
    Interface proxy = (Interface)Proxy.newProxyInstance(
      Interface.class.getClassLoader(),
      new Class[]{ Interface.class },
      new DynamicProxyHandler(real));
    consumer(proxy);
  }
} /* Output: (95% match)
doSomething
somethingElse bonobo
**** proxy: class $Proxy0, method: public abstract void
Interface.doSomething(), args: null
doSomething
**** proxy: class $Proxy0, method: public abstract void
Interface.somethingElse(java.lang.String), args:
[Ljava.lang.Object;@42e816
  bonobo
somethingElse bonobo
*///:~
```

595

通过调用静态方法**Proxy.newProxyInstance()**可以创建动态代理，这个方法需要得到一个类加载器（你通常可以从已经被加载的对象中获取其类加载器，然后传递给它），一个你希望该代理实现的接口列表（不是类或抽象类），以及**InvocationHandler**接口的一个实现。动态代理可以将所有调用重定向到调用处理器，因此通常会向调用处理器的构造器传递给一个“实际”对象的引用，从而使得调用处理器在执行其中介任务时，可以将请求转发。

invoke()方法中传递进来了代理对象，以防你需要区分请求的来源，但是在许多情况下，你并不关心这一点。然而，在**invoke()**内部，在代理上调用方法时需要格外当心，因为对接口的调用将被重定向为对代理的调用。

通常，你会执行被代理的操作，然后使用**Method.invoke()**将请求转发给被代理对象，并传入必需的参数。这初看起来可能有些受限，就像你只能执行泛化操作一样。但是，你可以通过传递其他的参数，来过滤某些方法调用：

```
//: typeinfo/SelectingMethods.java
// Looking for particular methods in a dynamic proxy.
import java.lang.reflect.*;
import static net.mindview.util.Print.*;

class MethodSelector implements InvocationHandler {
  private Object proxied;
  public MethodSelector(Object proxied) {
```

596

```
    this.proxied = proxied;
  }
  public Object
  invoke(Object proxy, Method method, Object[] args)
  throws Throwable {
    if(method.getName().equals("interesting"))
      print("Proxy detected the interesting method");
    return method.invoke(proxied, args);
  }
}

interface SomeMethods {
  void boring1();
  void boring2();
  void interesting(String arg);
  void boring3();
}

class Implementation implements SomeMethods {
  public void boring1() { print("boring1"); }
  public void boring2() { print("boring2"); }
  public void interesting(String arg) {
    print("interesting " + arg);
  }
  public void boring3() { print("boring3"); }
}

class SelectingMethods {
  public static void main(String[] args) {
    SomeMethods proxy= (SomeMethods)Proxy.newProxyInstance(
      SomeMethods.class.getClassLoader(),
      new Class[]{ SomeMethods.class },
      new MethodSelector(new Implementation()));
    proxy.boring1();
    proxy.boring2();
    proxy.interesting("bonobo");
    proxy.boring3();
  }
} /* Output:
boring1
boring2
Proxy detected the interesting method
interesting bonobo
boring3
*///:~
```

597

这里，我们只查看了方法名，但是你还可以查看方法签名的其他方面，甚至可以搜索特定的参数值。

动态代理并非是你日常使用的工具，但是它可以非常好地解决某些类型的问题。你在《Thinking in Patterns》（查看www.MindView.net）和Erich Gamma等人撰写的《Design Patterns》[⊖]这两本书中了解到有关代理和其他设计模式的更多知识。

练习21：(2) 修改**SimpleProxyDemo.java**，使其可以度量方法调用的次数。

练习22：(3) 修改**SimpleDynamicProxy.java**。使其可以度量方法调用的次数。

练习23：(3) 在**SimpleDynamicProxy.java**中的**invoke()**内部，尝试打印**proxy**参数并解释所产生的结果。

项目[⊜]：使用动态代理来编写一个系统以实现事务，其中，代理在被代理的调用执行成功时（不抛出任何异常）执行提交，而在其执行失败时执行回滚。你的提交和回滚都针对一个外

⊖ 本书的英文版、中文版及双语版均已由机械工业出版社出版。——编辑注

⊜ 项目实际上是被用作术语项目的一些建议，这些项目的解决方案并未包括在解决方案指南中。

部的文本文件，该文件不在Java异常的控制范围之内。你必须注意操作的原子性。

14.8 空对象

当你使用内置的**null**表示缺少对象时，在每次使用引用时都必须测试其是否为**null**，这显得枯燥，而且势必产生相当乏味的代码。问题在于**null**除了在你试图用它执行任何操作来产生**NullPointerException**之外，它自己没有其他任何行为。有时引入空对象[⊖]的思想将会很有用，它可以接受传递给它的所代表的对象的消息，但是将返回表示为实际上并不存在任何“真实”对象的值。通过这种方式，你可以假设所有的对象都是有效的，而不必浪费编程精力去检查**null**（并阅读所产生的代码）。 598

尽管想象一种可以自动为我们创建空对象的编程语言显得很有趣，但是实际上，到处使用空对象并没有任何意义——有时检查**null**就可以了，有时你可以合理地假设你跟本不会遇到**null**，有时甚至通过**NullPointerException**来探测异常也可以接受的。空对象最有用之处在于它更靠近数据，因为对象表示的是问题空间内的实体。有一个简单的例子，许多系统都有一个**Person**类，而在代码中，有很多情况是你没有一个实际的人（或者你有，但是你还没有这个人的全部信息），因此，通常你会使用一个**null**引用并测试它。与此不同的是，我们可以使用空对象。但是即使空对象可以响应“实际”对象可以响应的所有消息，你仍需要某种方式去测试其是否为空。要达到此目的，最简单的方式是创建一个标记接口：

```
//: net/mindview/util/Null.java
package net.mindview.util;
public interface Null {} ///:~
```

这使得**instanceof**可以探测空对象，更重要的是，这并不要求你在所有的类中都添加**isNull()**方法（毕竟，这只是执行RTTI的一种不同方式——为什么不使用内置的工具呢？）

```
//: typeinfo/Person.java
// A class with a Null Object.
import net.mindview.util.*;

class Person {
  public final String first;
  public final String last;
  public final String address;
  // etc.
  public Person(String first, String last, String address) {
    this.first = first;
    this.last = last;
    this.address = address;
  }
  public String toString() {
    return "Person: " + first + " " + last + " " + address;
  }
  public static class NullPerson
  extends Person implements Null {
    private NullPerson() { super("None", "None", "None"); }
    public String toString() { return "NullPerson"; }
  }
  public static final Person NULL = new NullPerson();
} ///:~
```

599

通常，空对象都是单例，因此这里将其作为静态**final**实例创建。这可以正常工作的，因为

⊖ 由Bobby Woolf和Bruce Anderson发现的。这可以看作是策略模式的特例。空对象的一种变体称为空迭代器模式，它使得在组合层次结构中遍历各个节点的操作对客户端透明（客户端可以使用相同的逻辑来遍历组合和叶子节点）。

Person是不可变的——你只能在构造器中设置它的值，然后读取这些值，但是你不能修改它们（因为**String**自身具备内在的不可变性）。如果你想要修改一个**NullPerson**，那只能用一个新的**Person**对象来替换它。注意，你可以选择使用**instanceof**来探测泛化的**Null**还是更具体的**NullPerson**，但是由于使用了单例方式，所以你还可以只使用**equals()**甚至**==**来与**Person.Null**比较。

现在假设你回到了互联网刚出现时的雄心万丈的年代，并且你已经因你惊人的理念而获得了一大笔的风险投资。你现在要招兵买马了，但是在虚位以待时，你可以将**Person**空对象放在每个**Position**上：

```
//: typeinfo/Position.java

class Position {
  private String title;
  private Person person;
  public Position(String jobTitle, Person employee) {
    title = jobTitle;
    person = employee;
    if(person == null)
      person = Person.NULL;
  }
  public Position(String jobTitle) {
    title = jobTitle;
    person = Person.NULL;
  }
  public String getTitle() { return title; }
  public void setTitle(String newTitle) {
    title = newTitle;
  }
  public Person getPerson() { return person; }
  public void setPerson(Person newPerson) {
    person = newPerson;
    if(person == null)
      person = Person.NULL;
  }
  public String toString() {
    return "Position: " + title + " " + person;
  }
} ///:~
```

600

有了**Position**，你就不需要创建空对象了，因为**Person.Null**的存在就表示这是一个空**Position**[稍后，你可能会发现需要增加一个显式的用于**Position**的空对象，但是YAGNI[⊖]（You Aren't Going to Need It，你永不需要它）声明：在你的设计草案的初稿中，应该努力使用最简单且可以工作的事物，直至程序的某个方面要求你添加额外的特性，而不是一开始就假设它是必需的]。

Staff类现在可以在你填充职位时查询空对象：

```
//: typeinfo/Staff.java
import java.util.*;

public class Staff extends ArrayList<Position> {
  public void add(String title, Person person) {
    add(new Position(title, person));
  }
  public void add(String... titles) {
    for(String title : titles)
      add(new Position(title));
  }
  public Staff(String... titles) { add(titles); }
  public boolean positionAvailable(String title) {
```

⊖ 这是极限编程（XP）的原则之一，即“做可以工作的最简单的事情”。

```
    for(Position position : this)
      if(position.getTitle().equals(title) &&
         position.getPerson() == Person.NULL)
        return true;
    return false;
  }
  public void fillPosition(String title, Person hire) {
    for(Position position : this)
      if(position.getTitle().equals(title) &&
         position.getPerson() == Person.NULL) {
        position.setPerson(hire);
        return;
      }
    throw new RuntimeException(
      "Position " + title + " not available");
  }
  public static void main(String[] args) {
    Staff staff = new Staff("President", "CTO",
      "Marketing Manager", "Product Manager",
      "Project Lead", "Software Engineer",
      "Software Engineer", "Software Engineer",
      "Software Engineer", "Test Engineer",
      "Technical Writer");
    staff.fillPosition("President",
      new Person("Me", "Last", "The Top, Lonely At"));
    staff.fillPosition("Project Lead",
      new Person("Janet", "Planner", "The Burbs"));
    if(staff.positionAvailable("Software Engineer"))
      staff.fillPosition("Software Engineer",
        new Person("Bob", "Coder", "Bright Light City"));
    System.out.println(staff);
  }
} /* Output:
[Position: President Person: Me Last The Top, Lonely At,
Position: CTO NullPerson, Position: Marketing Manager
NullPerson, Position: Product Manager NullPerson, Position:
Project Lead Person: Janet Planner The Burbs, Position:
Software Engineer Person: Bob Coder Bright Light City,
Position: Software Engineer NullPerson, Position: Software
Engineer NullPerson, Position: Software Engineer
NullPerson, Position: Test Engineer NullPerson, Position:
Technical Writer NullPerson]
*///:~
```

601

注意，你在某些地方仍必须测试空对象，这与检查是否为**null**没有差异，但是在其他地方（例如本例中的**toString()**转换）你就不必执行额外的测试了，而可以直接假设所有对象都是有效的。

如果你用接口取代具体类，那么就可以使用**DynamicProxy**来自动地创建空对象。假设我们有一个**Robot**接口，它定义了一个名字、一个模型和一个描述**Robot**行为能力的**List<Operation>**。**Operation**包含一个描述和一个命令（这是一种命令模式类型）：

602

```
//: typeinfo/Operation.java

public interface Operation {
  String description();
  void command();
} ///:~
```

你可以通过调用operations()来访问Robot的服务：

```
//: typeinfo/Robot.java
import java.util.*;
import net.mindview.util.*;

public interface Robot {
```

```
    String name();
    String model();
    List<Operation> operations();
    class Test {
      public static void test(Robot r) {
        if(r instanceof Null)
          System.out.println("[Null Robot]");
        System.out.println("Robot name: " + r.name());
        System.out.println("Robot model: " + r.model());
        for(Operation operation : r.operations()) {
          System.out.println(operation.description());
          operation.command();
        }
      }
    }
  } ///:~
```

这里也使用了嵌套类来执行测试。

我们现在可以创建一个扫雪Robot：

```
//: typeinfo/SnowRemovalRobot.java
import java.util.*;

public class SnowRemovalRobot implements Robot {
  private String name;
  public SnowRemovalRobot(String name) {this.name = name;}
  public String name() { return name; }
  public String model() { return "SnowBot Series 11"; }
  public List<Operation> operations() {
    return Arrays.asList(
      new Operation() {
        public String description() {
          return name + " can shovel snow";
        }
        public void command() {
          System.out.println(name + " shoveling snow");
        }
      },
      new Operation() {
        public String description() {
          return name + " can chip ice";
        }
        public void command() {
          System.out.println(name + " chipping ice");
        }
      },
      new Operation() {
        public String description() {
          return name + " can clear the roof";
        }
        public void command() {
          System.out.println(name + " clearing roof");
        }
      }
    );
  }
  public static void main(String[] args) {
    Robot.Test.test(new SnowRemovalRobot("Slusher"));
  }
} /* Output:
Robot name: Slusher
Robot model: SnowBot Series 11
Slusher can shovel snow
Slusher shoveling snow
Slusher can chip ice
Slusher chipping ice
Slusher can clear the roof
```

603

```
Slusher clearing roof
*///:~
```

604

假设存在许多不同类型的**Robot**，我们想对每一种**Robot**类型都创建一个空对象，去执行某些特殊操作——在本例中，即提供空对象所代表的**Robot**确切类型的信息。这些信息是通过动态代理捕获的：

```
//: typeinfo/NullRobot.java
// Using a dynamic proxy to create a Null Object.
import java.lang.reflect.*;
import java.util.*;
import net.mindview.util.*;

class NullRobotProxyHandler implements InvocationHandler {
  private String nullName;
  private Robot proxied = new NRobot();
  NullRobotProxyHandler(Class<? extends Robot> type) {
    nullName = type.getSimpleName() + " NullRobot";
  }
  private class NRobot implements Null, Robot {
    public String name() { return nullName; }
    public String model() { return nullName; }
    public List<Operation> operations() {
      return Collections.emptyList();
    }
  }
  public Object
  invoke(Object proxy, Method method, Object[] args)
  throws Throwable {
    return method.invoke(proxied, args);
  }
}

public class NullRobot {
  public static Robot
  newNullRobot(Class<? extends Robot> type) {
    return (Robot)Proxy.newProxyInstance(
      NullRobot.class.getClassLoader(),
      new Class[]{ Null.class, Robot.class },
      new NullRobotProxyHandler(type));
  }
  public static void main(String[] args) {
    Robot[] bots = {
      new SnowRemovalRobot("SnowBee"),
      newNullRobot(SnowRemovalRobot.class)
    };
    for(Robot bot : bots)
      Robot.Test.test(bot);
  }
} /* Output:
Robot name: SnowBee
Robot model: SnowBot Series 11
SnowBee can shovel snow
SnowBee shoveling snow
SnowBee can chip ice
SnowBee chipping ice
SnowBee can clear the roof
SnowBee clearing roof
[Null Robot]
Robot name: SnowRemovalRobot NullRobot
Robot model: SnowRemovalRobot NullRobot
*///:~
```

605

无论何时，如果你需要一个空**Robot**对象，只需调用**newNullRobot()**，并传递需要代理的**Robot**的类型。代理会满足**Robot**和**Null**接口的需求，并提供它所代理的类型的确切名字。

14.8.1 模拟对象与桩

空对象的逻辑变体是模拟对象和桩。与空对象一样，它们都表示在最终的程序中所使用的“实际”对象。但是，模拟对象和桩都只是假扮可以传递实际信息的存活对象，而不是像空对象那样可以成为**null**的一种更加智能化的替代物。

模拟对象和桩之间的差异在于程度不同。模拟对象往往是轻量级和自测试的，通常很多模拟对象被创建出来是为了处理各种不同的测试情况。桩只是返回桩数据，它通常是重量级的，并且经常在测试之间被复用。桩可以根据它们被调用的方式，通过配置进行修改，因此桩是一种复杂对象，它要做很多事。然而对于模拟对象，如果你需要做很多事情，通常会创建大量小而简单的模拟对象。

606 **练习24**：(4) 在**RegisteredFactories.java**中添加空对象。

14.9 接口与类型信息

interface关键字的一种重要目标就是允许程序员隔离构件，进而降低耦合性。如果你编写接口，那么就可以实现这一目标，但是通过类型信息，这种耦合性还是会传播出去——接口并非是对解耦的一种无懈可击的保障。下面有一个示例，先是一个接口：

```
//: typeinfo/interfacea/A.java
package typeinfo.interfacea;

public interface A {
  void f();
} ///:~
```

然后实现这个接口，你可以看到其代码是如何围绕着实际的实现类型潜行的：

```
//: typeinfo/InterfaceViolation.java
// Sneaking around an interface.
import typeinfo.interfacea.*;

class B implements A {
  public void f() {}
  public void g() {}
}

public class InterfaceViolation {
  public static void main(String[] args) {
    A a = new B();
    a.f();
    // a.g(); // Compile error
    System.out.println(a.getClass().getName());
    if(a instanceof B) {
      B b = (B)a;
      b.g();
    }
  }
} /* Output:
B
*///:~
```

通过使用RTTI，我们发现**a**是被当作**B**实现的。通过将其转型为**B**，我们可以调用不在**A**中的方法。

607

这是完全合法和可接受的，但是你也许并不想让客户端程序员这么做，因为这给了他们一个机会，使得他们的代码与你的代码的耦合程度超过你的期望。也就是说，你可能认为**interface**关键字正在保护着你，但是它并没有，在本例中使用**B**来实现**A**这一事实是公开有案

可查的[⊖]。

一种解决方案是直接声明，如果程序员决定使用实际的类而不是接口，他们需要自己对自己负责。这在很多情况下可能都是合理的，但“可能”还不够，你也许希望应用一些更严苛的控制。

最简单的方式是对实现使用包访问权限，这样在包外部的客户端就不能看到它了：

```
//: typeinfo/packageaccess/HiddenC.java
package typeinfo.packageaccess;
import typeinfo.interfacea.*;
import static net.mindview.util.Print.*;

class C implements A {
  public void f() { print("public C.f()"); }
  public void g() { print("public C.g()"); }
  void u() { print("package C.u()"); }
  protected void v() { print("protected C.v()"); }
  private void w() { print("private C.w()"); }
}

public class HiddenC {
  public static A makeA() { return new C(); }
} ///:~
```

在这个包中唯一**public**的部分，即**HiddenC**，在被调用时将产生**A**接口类型的对象。这里有趣之处在于：即使你从**makeA()**返回的是**C**类型，你在包的外部仍旧不能使用**A**之外的任何方法，因为你不能在包的外部命名**C**。

608

现在如果你试图将其向下转型为**C**，则将被禁止，因为在包的外部没有任何**C**类型可用：

```
//: typeinfo/HiddenImplementation.java
// Sneaking around package access.
import typeinfo.interfacea.*;
import typeinfo.packageaccess.*;
import java.lang.reflect.*;

public class HiddenImplementation {
  public static void main(String[] args) throws Exception {
    A a = HiddenC.makeA();
    a.f();
    System.out.println(a.getClass().getName());
    // Compile error: cannot find symbol 'C':
    /* if(a instanceof C) {
      C c = (C)a;
      c.g();
    } */
    // Oops! Reflection still allows us to call g():
    callHiddenMethod(a, "g");
    // And even methods that are less accessible!
    callHiddenMethod(a, "u");
    callHiddenMethod(a, "v");
    callHiddenMethod(a, "w");
  }
  static void callHiddenMethod(Object a, String methodName)
  throws Exception {
    Method g = a.getClass().getDeclaredMethod(methodName);
    g.setAccessible(true);
    g.invoke(a);
  }
```

⊖ 这种情况最出名的案例就是Windows操作系统，它有一个发布的API，并假设你会对着它进行编码，另外还有一个未发布的，但是可视的函数集，你可以发现并调用它。为了解决问题，程序员会使用隐藏的API函数，这导致微软必须把它们当作公共API的一部分来维护。因而成为了公司巨额成本和投入的黑洞。

```
} /* Output:
public C.f()
typeinfo.packageaccess.C
public C.g()
package C.u()
protected C.v()
private C.w()
*///:~
```

正如你所看到的，通过使用反射，仍旧可以到达并调用所有方法，甚至是**private**方法！如果知道方法名，你就可以在其**Method**对象上调用**setAccessible(true)**，就像在**callHiddenMethod()**中看到的那样。

609

你可能会认为，可以通过只发布编译后的代码来阻止这种情况，但是这并不解决问题。因为只需运行**javap**，一个随JDK发布的反编译器即可突破这一限制。下面是一个使用它的命令行：

```
javap -private C
```

-private标志表示所有的成员都应该显示，甚至包括私有成员。下面是输出：

```
class typeinfo.packageaccess.C extends
java.lang.Object implements typeinfo.interfacea.A {
    typeinfo.packageaccess.C();
    public void f();
    public void g();
    void u();
    protected void v();
    private void w();
}
```

因此任何人都可以获取你最私有的方法的名字和签名，然后调用它们。

如果你将接口实现为一个私有内部类，又会怎样呢？下面展示了这种情况：

```
//: typeinfo/InnerImplementation.java
// Private inner classes can't hide from reflection.
import typeinfo.interfacea.*;
import static net.mindview.util.Print.*;

class InnerA {
  private static class C implements A {
    public void f() { print("public C.f()"); }
    public void g() { print("public C.g()"); }
    void u() { print("package C.u()"); }
    protected void v() { print("protected C.v()"); }
    private void w() { print("private C.w()"); }
  }
  public static A makeA() { return new C(); }
}

public class InnerImplementation {
  public static void main(String[] args) throws Exception {
    A a = InnerA.makeA();
    a.f();
    System.out.println(a.getClass().getName());
    // Reflection still gets into the private class:
    HiddenImplementation.callHiddenMethod(a, "g");
    HiddenImplementation.callHiddenMethod(a, "u");
    HiddenImplementation.callHiddenMethod(a, "v");
    HiddenImplementation.callHiddenMethod(a, "w");
  }
} /* Output:
public C.f()
InnerA$C
public C.g()
package C.u()
protected C.v()
```

610

```
private C.w()
*///:~
```

这里对反射仍旧没有隐藏任何东西。那么如果是匿名类呢?

```
//: typeinfo/AnonymousImplementation.java
// Anonymous inner classes can't hide from reflection.
import typeinfo.interfacea.*;
import static net.mindview.util.Print.*;

class AnonymousA {
  public static A makeA() {
    return new A() {
      public void f() { print("public C.f()"); }
      public void g() { print("public C.g()"); }
      void u() { print("package C.u()"); }
      protected void v() { print("protected C.v()"); }
      private void w() { print("private C.w()"); }
    };
  }
}

public class AnonymousImplementation {
  public static void main(String[] args) throws Exception {
    A a = AnonymousA.makeA();
    a.f();
    System.out.println(a.getClass().getName());
    // Reflection still gets into the anonymous class:
    HiddenImplementation.callHiddenMethod(a, "g");
    HiddenImplementation.callHiddenMethod(a, "u");
    HiddenImplementation.callHiddenMethod(a, "v");
    HiddenImplementation.callHiddenMethod(a, "w");
  }
} /* Output:
public C.f()
AnonymousA$1
public C.g()
package C.u()
protected C.v()
private C.w()
*///:~
```

611

看起来没有任何方式可以阻止反射到达并调用那些非公共访问权限的方法。对于域来说，的确如此，即便是**private**域：

```
//: typeinfo/ModifyingPrivateFields.java
import java.lang.reflect.*;

class WithPrivateFinalField {
  private int i = 1;
  private final String s = "I'm totally safe";
  private String s2 = "Am I safe?";
  public String toString() {
    return "i = " + i + ", " + s + ", " + s2;
  }
}

public class ModifyingPrivateFields {
  public static void main(String[] args) throws Exception {
    WithPrivateFinalField pf = new WithPrivateFinalField();
    System.out.println(pf);
    Field f = pf.getClass().getDeclaredField("i");
    f.setAccessible(true);
    System.out.println("f.getInt(pf): " + f.getInt(pf));
    f.setInt(pf, 47);
    System.out.println(pf);
    f = pf.getClass().getDeclaredField("s");
```

```
        f.setAccessible(true);
        System.out.println("f.get(pf): " + f.get(pf));
        f.set(pf, "No, you're not!");
        System.out.println(pf);
        f = pf.getClass().getDeclaredField("s2");
        f.setAccessible(true);
        System.out.println("f.get(pf): " + f.get(pf));
        f.set(pf, "No, you're not!");
        System.out.println(pf);
      }
    } /* Output:
    i = 1, I'm totally safe, Am I safe?
    f.getInt(pf): 1
    i = 47, I'm totally safe, Am I safe?
    f.get(pf): I'm totally safe
    i = 47, I'm totally safe, Am I safe?
    f.get(pf): Am I safe?
    i = 47, I'm totally safe, No, you're not!
    *///:~
```

612

但是，**final**域实际上在遭遇修改时是安全的。运行时系统会在不抛异常的情况下接受任何修改尝试，但是实际上不会发生任何修改。

通常，所有这些违反访问权限的操作并非世上最遭之事。如果有人使用这样的技术去调用标识为**private**或包访问权限的方法（很明显这些访问权限表示这些人不应该调用它们），那么对他们来说，如果你修改了这些方法的某些方面，他们不应该抱怨。另一方面，总是在类中留下后门的这一事实，也许可以使得你能够解决某些特定类型的问题，但如果不这样做，这些问题将难以或者不可能解决，通常反射带来的好处是不可否认的。

练习25：(2) 创建一个包含**private**、**protected**和包访问权限方法的类，编写代码在该类所处的包的外部调用访问这些方法。

14.10 总结

RTTI允许通过匿名基类的引用来发现类型信息。初学者极易误用它，因为在学会使用多态调用方法之前，这么做也很有效。对有过程化编程背景的人来说，很难让他们不把程序组织成一系列**switch**语句。你可以用RTTI做到这一点，但是这样就在代码开发和维护过程中损失了多态机制的重要价值。面向对象编程语言的目的是让我们在凡是可以使用的地方都使用多态机制，只在必需的时候使用RTTI。

613

然而使用多态机制的方法调用，要求我们拥有基类定义的控制权，因为在你扩展程序的时候，可能会发现基类并未包含我们想要的方法。如果基类来别人的类，或者由别人控制，这时候RTTI便是一种解决之道：可继承一个新类，然后添加你需要的方法。在代码的其他地方，可以检查你自己特定的类型，并调用你自己的方法。这样做不会破坏多态性以及程序的扩展能力，因为这样添加一个新的类并不需要在程序中搜索**switch**语句。但如果在程序主体中添加需要的新特性的代码，就必须使用RTTI来检查你的特定的类型。

如果只是为了某个特定类的利益，而将某个特性放进基类里，这意味着从那个基类派生出的所有其他子类都带有这些可能无意义的东西。这会使得接口更不清晰，因为我们必须覆盖由基类继承而来的所有抽象方法，这是很恼人的。例如，考虑一个表示乐器**Instrument**的类层次结构。假设我们想清洁管弦乐队中某些乐器残留的口水，一种办法是在基类**Instrument**中放入**clearSpitValve()**方法。但这样做会造成混淆，因为它意味着打击乐器**Percussion**、弦乐器**Stringed**和电子乐器**Electronic** 也需要清洁口水。在这个例子中，RTTI可以提供了一种更合理的解决方案。可以将**clearSpitValve()**置于适当的特定类中，在这个例子中是**Wind**（管乐器）。同时，

你可能会发现还有更恰当的解决方法，在这里，就是将**prepareInstrument()**置于基类中，但是初次面对这个问题时读者可能看不到这样的解决方案，而误认为必须使用RTTI。

最后一点，RTTI有时能解决效率问题。也许你的程序漂亮地运用了多态，但其中某个对象是以极端缺乏效率的方式达到这个目的的。你可以挑出这个类，使用RTTI，并且为其编写一段特别的代码以提高效率。然而必须要注意，不要太早地关注程序的效率问题，这是个诱人的陷阱。最好首先让程序运作起来，然后再考虑它的速度，如果要解决效率问题可以使用profiler（查看http://MindView.net/Books/BetterJava上的补充材料）。

我们已经看到了，由于反射允许更加动态的编程风格，因此它开创了编程的新世界。对有些人来说，反射的动态特性是一种烦扰，对于已经习惯于静态类型检查的安全性的人来说，你可以执行一些只能在运行时进行的检查，并用异常来报告检查结果的行为，这本身就是一种错误的方向。有些人走的更远，他们声称引入运行时异常本身就是一种指示，说明应该避免这种代码。我发现这种意义的安全是一种错觉，因为总是有些事情是在运行时发生并抛出异常的，即使是在不包含任何**try**语句块或异常规格说明的程序中也是如此。因此，我认为一致的错误报告模型的存在使我们能够通过使用反射编写动态代码。当然，尽力编写能够进行静态检查的代码是值得的，只要你确实能够这么做。但是我相信动态代码是将Java与其他诸如C++这样的语言区分开的重要工具之一。 614

练习26：(3) 实现本章总结中所描述的**clearSpitValve()**。

所选习题的答案都可以在名为The Thinking in Java Annotated Solution Guide的电子文档中找到，读者可以从www.MindView.net购买此文档。 615 ~ 616